

Interactive Population Visualization of Virginia Using VTK and Vega-Lite in Trame

Jayachandra Sarika
Department of Computer Science
University of Houston, Houston, TX, USA
jsarika@cougarnet.uh.edu

Anjani Kumar Avadhanam
Department of Computer Science
University of Houston, Houston, TX, USA
aavadhan@cougarnet.uh.edu

Minh Nguyen
Department of Computer Science
University of Houston, Houston, TX, USA
mdnguy31@cougarnet.uh.edu

Abstract— Understanding how populations changes across different geographic regions requires visualization tools that connect both spatial and temporal information. In this project we developed an interactive, web based system that allows users to explore population trends for Virginia counties between 2010 and 2019. The system combines VTK for efficient rendering of the county map with Vega Lite for generating a range of analytical charts, all integrated through the Trame framework to provide an efficient browser interface. Our application supports several forms of analysis like including population trends over time, summary statistics, similarity detection between counties, two-county comparisons, year by year growth patterns, forecasting using linear regression, K-Means clustering to uncover broader demographic patterns. main feature of the system is the two side interaction between the VTK map and the Vega lite charts by selecting a county updates all analytical views, while changes in the analytical panels maintain the spatial context. Overall, this work demonstrates how combining VTK’s geographic visualization capabilities with the flexibility of creative charts can create an effective, scalable tool for demographic exploration. The system highlights how modern visualization frameworks can be integrated to support reproducible, interactive, and data driven analysis.

Index Terms—Interactive Visualization, Trame, Vega Lite, VTK, Demographic Analysis, Geographic Information Visualization, Population Trends and Visual Analytics.

1 INTRODUCTION

Studying about population grows or declines in different areas is important for many fields like including planning cities, improving infrastructure, understanding social and economic changes. To do this effectively, analysts need tools that help them see not only the numbers, but also how those numbers change over time and locations. Traditional charts and static maps are often not enough because they cannot show these patterns together. They also limit the user’s ability to explore data interactively, compare multiple regions in a flexible way, or uncover deeper trends that only become visible when different visual views and analytical methods work together.

Modern visualization tools enable interactive web systems that combine fast VTK-based map rendering with clear Vega-Lite analytical charts. Trame links the backend, VTK view, and browser charts, allowing users to explore both spatial and analytical data in a synchronized way.

This project presents an interactive system for exploring population changes across Virginia’s counties from 2010–2019. A VTK-based map and Vega-Lite charts let users examine trends, summaries, comparisons, forecasts, and clustering, with two-way interaction keeping spatial and analytical views synchronized.

Building such a system is challenging because VTK and Vega-Lite use different data models and interaction styles. Coordinating them requires shared state, smooth event handling, and real-time updates. With Trame’s effective design, we implemented a unified and responsive environment for demographic exploration.

Overall, this project shows that using VTK for spatial maps together with Vega-Lite for analytical charts can build a strong and effective tool for exploring population data. Our system demonstrates the value of linking multiple views, using a modular data pipeline, and supporting

interactive analysis. It also tells how different visualization frameworks can work together within one browser based platform.

2 RELATED WORK

Our system brings several areas together : scientific visualization tools, high-level visualization grammars, interactive web based analytics, demographic data exploration. In this section, we review the most relevant work in each of these fields and explain how our approach builds on them and connects these different ideas into a single system.

The Visualization Toolkit (VTK) has long been central to scientific visualization and 3D graphics. Schroeder et al. describe it as a modular, object-oriented system for building pipelines and rendering polygonal data [5]. We use VTK in the same spirit to represent Virginia’s county polygons, store per-cell attributes, and render the map in the browser through Trame. Recent work such as VTK-m extends these ideas to large-scale parallel computing [3]. Although our system is smaller, it benefits from VTK’s clear separation of data processing and rendering, supporting responsive, interactive analytics.

VTK has long been a core toolkit for scientific visualization. Schroeder et al.its modular pipeline for rendering polygonal data [5]. We use VTK to model Virginia’s county polygons, store per-cell attributes, and render the map through Trame. Newer systems like VTK-m extend these ideas to large-scale parallel settings [3]. Even at our smaller scale, VTK’s separation of data processing and rendering supports fast, interactive analytics.

Clustering and other unsupervised learning methods are widely used in visual analytics to uncover hidden patterns in complex datasets. Han et al. describe K-Means in detail, including how distances are measured, how data should be normalized, and how cluster centers can be interpreted within data mining [1]. Following this approach, we normalize each county’s population trend and apply K-Means to group counties with similar patterns over time. These clusters are then shown in two ways: as a Vega-Lite scatterplot in attribute space and as color-coded regions on the VTK map. Combining algorithmic grouping with linked visual views helps reveal structure that would be hard to

notice in raw data or a single chart.

Our work also relates to systems that use multiple coordinated views for geographic and time-based data. Many demographic dashboards link choropleth maps with time-series charts, allowing basic brushing and highlighting across views. However, these tools are usually built on a single web-graphics stack and do not take advantage of high-performance systems like VTK. In contrast our idea combines a scientific visualization toolkit with a high level charting grammar. Here all connected through a reactive state management layer. This setup keeps geographic interactions and analytical views tightly coupled and still allows flexible and expressive chart design.

In summary, our project builds on VTKs long history in scientific visualization [3, 5], draws from high level visualization grammars such as Vega Lite [2, 4, 6], and uses standard K-Means clustering methods from data mining [1]. What makes our work different is how we bring these pieces together into one browser for analyzing county level population data. By combining spatial rendering, declarative charting, interactive clustering, we create a single integrated environment for demographic exploration.

3 DATASET

Our system is built on two primary data sources: a tabular population dataset and a polygonal geographic dataset representing the boundaries of all Virginia counties together, these datasets enable the integration of spatial, temporal, and statistical information within an interactive visual analytical environment.

3.1 Population Dataset

We use a county level population dataset containing annual population estimates for the Commonwealth of Virginia from year 2010 to 2019. The dataset was given a CSV file named `virginia_population.csv`, consisting of 134 rows and 11 columns. Each row corresponds to one of Virginia's 133 counties or independent cities with an additional header row. These columns include the county name followed by ten numerical fields representing population counts for each year from 2010 to 2019.

The structure of the dataset is as follows:

- **County:** Name of the county or independent city.
- **2010–2019:** Integer population estimates for every year.

The dataset provides ten years of population information, which allows for analyzing such as trend evaluation, percentage growth, and year by year comparisons. Only minimal cleaning was needed, mainly standardizing county names and making sure all population values were stored as integers. Since there were no missing values, the preprocessing process was straightforward.

3.2 Geographic Dataset

To visualize counties spatially, we use a VTK polygonal mesh representing the boundaries of all 133 Virginia counties. This mesh is stored as a `vtkPolyData` object containing:

- A set of 2D polygonal cells corresponding to county outlines.
- Vertex coordinates defining the geographic shape of each county.
- Connectivity information mapping points to polygonal faces.

Each polygon represents a county, but some counties have multiple separated regions, so one county can correspond to several VTK cell IDs. During preprocessing, we matched each county name to its set of polygon cells, creating a lookup structure that lets the system color and highlight counties interactively.

3.3 Data Integration and Preprocessing

Here the two datasets came from different sources and used different naming formats, we needed to normalize the names to match population rows with the geographic polygons. To do this, we created a name-normalization function that standardizes the county names.

- Converts names to lowercase,
- Removes punctuation such as hyphens and apostrophes,

- Standardizes spacing and suffixes (e.g., “County”, “City”).

This ensures each row in the population CSV linked to one or more polygon cells in the VTK mesh. A dictionary-style index, `COUNTY_INDEX`, maps normalized county names to:

- the row number in the population table,
- the list of VTK cell IDs forming the county's geometry.

This integration is essential for coordinating interactions: choosing a county on the map updates all Vega Lite charts and selecting a county from the dropdown highlights the correct regions in the VTK map.

3.4 Derived Attributes

In addition to raw population values, several derived fields were computed to support the analysis:

- **Population change (2010–2019):** Absolute and percentage change.
- **Rankings:** Population rank and growth rank across the counties.
- **Normalized population vectors:** Used for similarity scoring and K-Means clustering.
- **County area (optional):** Derived from the polygonal surface area for experimental features.

These derived values support advanced visualizations like similarity charts, growth rankings, correlation matrices, forecasts, clustering plots together, these population data and geographic mesh create a strong base for linked spatial and temporal analysis. The clean CSV structure and flexible VTK polygons makes it easy to combine numerical analysis with geographic visualization in our Trame based system.

4 METHODOLOGY AND IMPLEMENTATION

This section explains us how we designed and build our interactive system for exploring county level population trends in Virginia. Our aim is to create a unified workflow that connects data loading, backend logic, fast geographic rendering, clear analytical charts in one browser interface. To do this we use three main technologies: VTK for rendering the county mesh and handling cell colors [5], Trame for managing the web application state, Vega-Altair (compiled to Vega Lite) for generating interactive charts [4]. The system relies on a reactive state model where any user action such as selecting a county or switching tabs updates shared state and triggers synchronized changes in both the VTK map and the analytical views.

The system is modular Architecture. Here the data layer loads the population CSV and VTK mesh, matches the county identifiers and computes derived values like growth and similarity. Above this a Trame based layer manages the server side state and provides controller functions that update both the VTK map colors and the Vega Altair charts. The interface is divided into several tabs main analysis, population share, yearly bars, top growth, forecasting, correlation, comparison, clustering each supported by its own chart building function to keep the visualizations consistent and easy to maintain.

We also add the lightweight machine learning methods to enhance the analysis. K-Means clustering groups the counties based on the normalized population trends and a simple linear regression model predicts future values for each county [1]. These models are not treated as black boxes; their results are directly shown through clear charts and color coded map views. The interaction design ensures smooth coordination selecting a county updates all charts, switching tabs changes the analytical view, and enabling comparison or clustering adjusts the visual encoding to highlight these modes.

4.1 Overview of the System Architecture

Our visualization system uses a layered client server design that links data processing, rendering and the interactive UI components through the trame framework. Figure X shows a high level view of the main modules and how they interact each other.

On top of the data layer, an analytics and visualization layer handles all non-spatial computations and chart creation. In `charts.py`, we define Altair/Vega-Lite specifications for population trends, yearly bar

charts, population share, similarity bars, growth gauges, K-Means clustering (with PCA), and a basic linear-regression forecast. These functions work only with tabular data and return declarative chart objects that can be rendered in the browser.

The rendering layer handles the geographic view using VTK. The `map_utils.py` module sets up a `vtkPolyDataMapper`, a map actor, and a per-cell color array for highlighting counties. It also provides utility functions to reset the map to a default color and to highlight a selected county by updating the color array.

At the top level, the `trame` server in `app.py` manages state and user interaction. The server defines reactive state variables (such as `selected_county`, `compare_mode`, and `cluster_k`) and callback functions that update both the VTK map and the Vega-Altair charts when the state changes. The user interface, built with Vuetify components, is rendered in the browser and arranged as a left-hand VTK map panel and a right-hand panel of tabbed analytic views. User actions—such as selecting a county, toggling compare mode, or moving the cluster slider—update the `trame` state, trigger the appropriate callbacks, and send updated VTK images and JSON chart specifications back to the client. This design keeps data, analytics, rendering, and interaction separate while still maintaining tight synchronization between the map and the statistical views. .

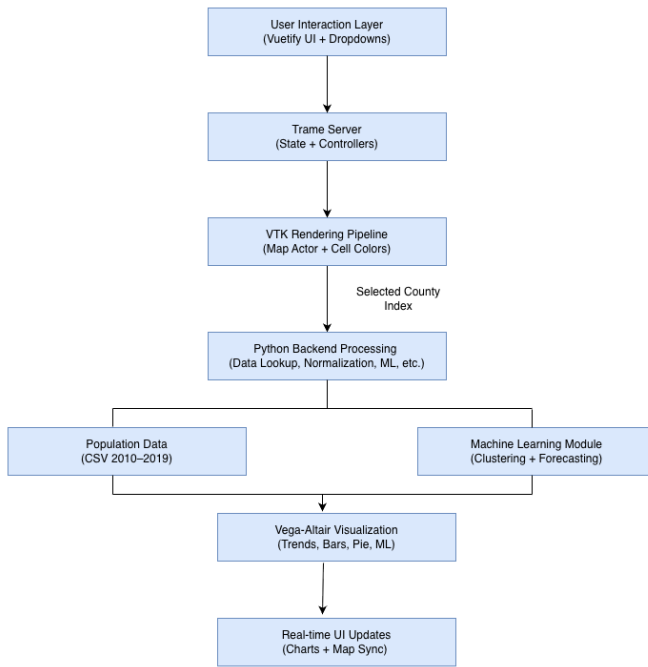


Fig. 1: High-level system flowchart showing the interactions between Trame, VTK, Vega-Altair, the Python backend, and the user interface.

4.2 VTK Rendering Pipeline for Geographic Data

The geographic visualization in our system is built using the Visualization Toolkit (VTK), which provides an efficient framework for rendering polygon-based spatial data. The Virginia county boundaries are stored as a `vtkPolyData` object, where each county may consist of one or more polygonal cells. To enable interactive recoloring and highlighting, we attach a cell-level `vtkUnsignedCharArray` that stores RGB values for each polygon in the mesh.

During initialization, the system executes the pipeline defined in `map_utils.create_virginia_actor()`, which constructs the core rendering objects:

1. A **color array** initialized to a uniform base color (RGB = (235, 235, 235)).
2. A **vtkPolyDataMapper** configured to use the cell scalars for direct color rendering.

3. A **vtkActor** with customized polygon edge styling, including dark grey boundaries and increased line thickness to enhance map readability.

The actor is added to a VTK renderer with parallel projection enabled, which makes the visualization appear as a flat 2D map even though it is drawn in a 3D graphics environment. The renderer is integrated into the Trame framework using a `VtkRemoteView`, which streams rendered frames to the browser and updates whenever the cell colors change. For interactivity, we use a simple highlighting function, `highlight_county()`, which resets all cells to the base color and then recolors only the selected county's polygons. Because Trame handles view invalidation automatically, calling `colors.Modified()` is enough to refresh the map in the browser.

Pseudocode: VTK Highlighting Procedure

```

function highlight_county(poly, colors, cell_ids):
    # Reset entire map to base color
    for cid in range(poly.num_cells):
        colors[cid] = BASE_COLOR

    # Apply highlight color to selected polygons
    for cid in cell_ids:
        colors[cid] = HIGHLIGHT_COLOR

    colors.Modified()
    update_view()
  
```

4.3 Interactive County Selection and Real-time Synchronization with Trame

County selection in our system is done through a dropdown menu in the Trame interface. When a user picks a county, the value is stored in the reactive `selected_county` state, and Trame automatically updates the VTK map and all Vega-Lite views to keep them synchronized.

When the state changes, the system uses a lookup table to find the selected county's CSV row and VTK cell IDs, updates the map by recoloring those cells, and streams the new frame to the browser through `VtkRemoteView`. At the same time, all analytical modules read the updated `selected_county` value, rebuild their Vega-Lite charts, and send the refreshed specifications to the browser through Trame.

Pseudocode: State-driven Update Workflow

```

on_state_change(selected_county):

    # 1) Update VTK map
    cell_ids = COUNTY_INDEX[selected_county].cell_ids
    highlight_county(poly, color_array, cell_ids)
    update_view()

    # 2) Update analytical charts
    row = COUNTY_INDEX[selected_county].csv_row
    update_trend_chart(DF, row)
    update_similarity_chart(DF, row)
    update_bar_2010_2019(DF, row)
    update_growth_gauge(DF, row)
    update_population_share(DF, row)
    update_yearly_bars(DF, row)
    update_correlation_heatmap(DF)
    update_forecast_plot(DF, row)
  
```

This unified event-driven mechanism ensures that all visual modules remain synchronized, producing a cohesive analytical environment where geographic and statistical views update in real time with minimal developer overhead.

4.4 Visualization of Statistical Data with Vega-Altair

To complement the VTK geographic view, our system uses the Vega-Altair grammar for statistical and time-based analysis. Altair provides clear declarative visualization specifications that work smoothly with

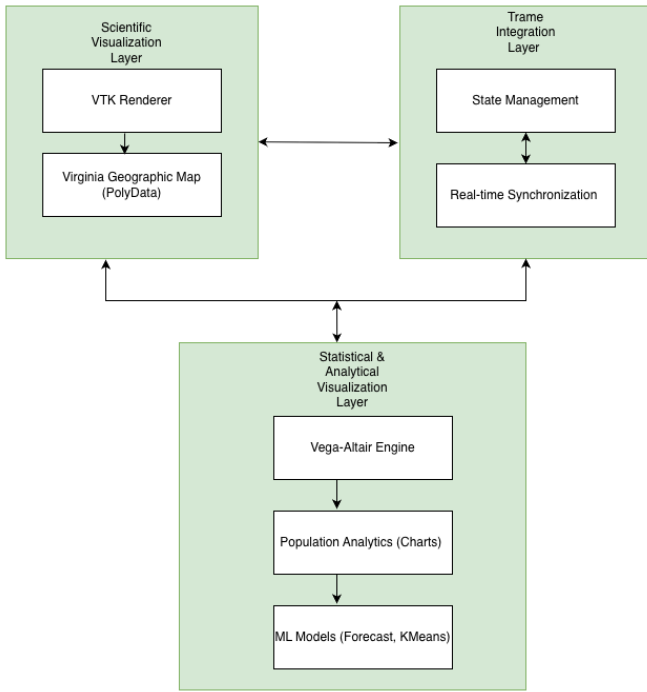


Fig. 2: System architecture showing how VTK geographic rendering, Vega-Altair statistical analysis, and machine-learning modules are coordinated through Trame’s state management and **Real-Time System Synchronization** layer.

Trame’s reactive UI. Each analytical panel is built as an Altair Chart, translated into a Vega-Lite specification, and rendered in the browser. Because Vega-Lite supports tooltips, legends, and interaction, users can explore demographic patterns through coordinated analytical views.

All statistical visualizations read from the unified population DataFrame and update whenever the selected county changes, keeping all analytical views insync with the VTK map. Below we summarize the main analytical modules in our system.

Population Trend Line Chart. This chart shows the selected county’s population from 2010–2019 using a line with points. It highlights growth or decline clearly, with a “nice” Y-axis scale for readability.

2010–2019 Mini Bar Comparison. A small horizontal bar chart compares the 2010 and 2019 population values, providing a quick summary of long-term change for the selected county.

Similarity Ranking of Counties. Standardized population vectors are used to compute Euclidean distances and find similar counties. The five closest matches appear in a bar chart with color indicating similarity strength.

Ten-Year Growth Gauge. Percentage population growth is shown with a semi-circular gauge using an arc background and a moving needle. The gauge is created from layered Altair marks, showing the flexibility of declarative chart composition.

Population Share Pie Chart. A donut chart shows the selected county’s share of Virginia’s 2019 population. This view offers us quick context especially for the counties with very large or very small population sizes.

Yearly Bar Chart. A ten-year bar chart displays the population for each year, giving a clear view of year-to-year changes. Altair tooltips let users check exact population values directly.

Growth Correlation Heatmap. We compute year-to-year population changes and measure correlations among all intervals. The results are shown as a red–blue heatmap, helping reveal temporal patterns such as consistent early or late growth.

Predictive Forecasting. A simple linear regression model forecasts population through 2030. Historical and predicted values are shown together in a two-color line chart, helping users spot accelerating or slowing growth trends.

Cluster Visualization (K-Means with PCA Projection). K-Means clustering groups counties based on normalized population trends, and a 2D PCA scatterplot displays these clusters. Tooltips show county names and cluster labels, making patterns easy to interpret.

Pseudocode: Vega-Altair Chart Update Flow

`on_selected_county_change(row_index):`

```

# Recompute dataset slice
pops = DF[year_cols].iloc[row_index]

# Build chart specifications
trend_chart = build_trend_chart(pops)
bar_chart = build_bar_chart(pops)
sim_chart = compute_similar_counties(DF, row_index)
gauge_chart = build_growth_gauge(pops)
share_chart = build_population_share(DF, row_index)
yearly_chart = build_yearly_bars(pops)
corr_heatmap = compute_growth_correlation(DF)
forecast_chart = linear_regression_forecast(pops)
cluster_chart = compute_kmeans_clustering(DF)

# Push to browser
ctrl.update_chart(trend_chart)
ctrl.update_mini_chart(bar_chart)
ctrl.update_similar_chart(sim_chart)
ctrl.update_gauge_chart(gauge_chart)
ctrl.update_pie_chart(share_chart)
ctrl.update_yearly_bars(yearly_chart)
ctrl.update_corr_chart(corr_heatmap)
ctrl.update_forecast_chart(forecast_chart)
ctrl.update_cluster_chart(cluster_chart)

```

Altogether, these statistical views provide a comprehensive analytical toolkit that enhances the geographic visualization, enabling users to conduct multi-faceted demographic exploration directly within the browser.

4.5 Machine Learning Integration: Clustering and Forecasting

Machine learning methods were added to provide higher-level analytical insight. We use K-Means clustering to reveal groups of similar counties and linear regression to forecast future population trends. Both models run on the population time-series data and update instantly through the Trame event loop.

4.5.1 K-Means Clustering of Population Trends

We applied K-Means clustering to the normalized population matrix, where each county is a ten-dimensional vector (2010–2019). Using scikit-learn’s StandardScaler standardizes the data so clustering reflects trend shape rather than population size.

```

# Pseudocode: K-Means clustering pipeline
X = DF[year_cols].astype(int).values
X_norm = StandardScaler().fit_transform(X)
clusters = KMeans(k).fit_predict(X_norm)
DF["Cluster"] = clusters

```

After clustering, we use PCA to reduce the ten-dimensional population vectors to two dimensions for visualization, showing each county as a color-coded point with tooltips for names and cluster labels. The

number of clusters k is controlled with a slider and all the views update instantly through the Trame.

4.5.2 Future Population Forecasting

To forecast future populations, we use a simple linear regression model that fits each county's historical values with the first order polynomial using NumPy's `polyfit`, producing a basic trend line for prediction.

$$\hat{P}(t) = \beta_0 + \beta_1 t$$

Here, t is the year, β_1 is the yearly change, and β_0 is the intercept; the model then predicts 2020–2030 values as a simple clear trend. Vega Altair visualizes this with two lines historical and predicted so the users can quickly see expected growth or decline.

```
# Pseudocode: Linear regression forecast
years = array([2010 ... 2019])
pops = DF.loc[row, year_cols]
coef = polyfit(years, pops, 1)
future_years = range(2020, 2031)
preds = polyval(coef)(future_years)
```

4.5.3 Integration with the Interactive Framework

Both machine learning modules are fully integrated with the Trame event-driven architecture. When users:

- changes the selected county,
- switches to the “Clustering” or “Forecast” tab or
- adjust the K-value slider

The system recomputes the models and updates the Vega Altair views in realtime letting the users adjust clusters explore similarities and examine forecasts without interrupting the workflow. Clustering and simple forecasting add analytical depth, supporting more meaningful exploration of demographic patterns across the Virginia.

4.6 Interaction Design and UI Layout

The system here follows a coordinated multiple views (CMV) design allowing users to move smoothly between spatial, statistical and analytical perspectives. The interface uses Trame's reactive state and Vuetify components, with a left panel showing the VTK map and the right panel that updates based on the selected analysis mode.

4.6.1 County Selection and Reactive Updates

Users begin exploration by choosing the county from the toolbar dropdown this triggers a Trame state update and synchronizes all linked views. The selected county's polygons are highlighted in the VTK map and all Vega Altair charts including trends, bars, summary, similarity and growth views are recomputed so that every visualization stays aligned with the users focus.

4.6.2 Tab-Based Analytical Views

The right side panel is structured as a tabbed layout with each tab dedicated to a specific analytic function. These include:

- **Main Analysis** – population trend, statistics, comparison, similarity and growth gauge.
- **Population Share** – a pie chart is showing us the county's share of the statewide population.
- **Yearly Bars** – a bar chart is summarizing annual population values.
- **Top Growth** – a leaderboard of the fastest-growing counties.
- **Predictive Forecast** – a regression-based future population projection.
- **Growth Correlation** – a heatmap displaying year-over-year growth correlations.
- **Clusters** – interactive K-Means clustering with adjustable cluster count.

Switching between tabs updates only the top-section state variable, ensuring the VTK map remains visible while the analytical content adapts accordingly.

4.6.3 Two-County Comparison Mode

A “Compare Counties” toggle activates a two-county comparison mode, switching the right panel to a line chart that shows both counties' population trends across the decade. Two dropdowns (County A and County B) let users choose targets, and Trame updates the view in real time; turning the mode off restores the default analysis view.

4.6.4 Interactive Clustering Control

In the Clusters tab, a slider adjusts the number of clusters k , updating the `cluster_k` state and triggering both a recalculation of K-Means and a recoloring of counties in the VTK map. A PCA scatterplot in the right panel gives an additional interactive view of how the clusters separate.

4.6.5 Consistent Visualization–Interaction Coupling

A main requirement is to provide constant behavior across all interactive elements. All county dependent charts respond to any of the following:

- a change in the selected county's
- activation of the specific analytic tab,
- updates from the cluster computation,
- toggling the comparison mode.

Trame's event driven model links callbacks to the state changes by keeping all components synchronized automatically. This ensures every view updates together without any manual refresh or page reload.

4.6.6 User-Centered Layout and Visual Design

The system uses a clear visual layout: the fixed VTK map provides spatial context while the right side panel show analytical views in a clean scroll free format. The Vega altair charts use consistent sizing and accessible color palettes to maintain readability. The overall design emphasizes clarity, responsiveness and smooth navigation so users can explore and compare demographic patterns easily.

4.7 Team Contributions

Each team member contributed equally to the development of this visualization system.

Jayachandra Sarika led the VTK pipeline development, including the conversion of the Virginia shapefile into the VTK polydata. The construction of the rendering and highlighting logic and creation of the main Trame user interface layout. He also integrated the interactive modules like similarity analysis, forecasting, correlation visualization and clustering.

Anjani Kumar Avadhanam focused on data processing and analytical visualization. He prepared the population dataset had implemented the Altair based charts, trend lines, yearly bars, growth leaderboard, mini comparisons and contributed to the similarity computation, forecasting the logic and compare mode interaction design.

Minh Nguyen contributed to system integration, UI refinement and advanced analytics. He developed the Principal Component Analysis based cluster visualization So the correlation heatmap, and the several Trame state management callbacks ensuring smooth interaction between the VTK map and Vega Lite charts. He also improved layout structure and consistency in different panels.

5 RESULTS AND DISCUSSION

The system provides us the coordinated views that reveal key demographic patterns across Virginia's 133 counties from year 2010 to 2019. Each visualization highlights the different aspect temporal change, spatial distribution, similarity, trends and forecasted growth letting the users explore the data from multiple angles.

5.1 Population Trend Exploration

Figure 3 shows the decade-long trend for Accomack County, which remains steady around 33,000–34,000 residents. This stability contrasts with fast-growing urban counties, helping users quickly spot slower-changing regions.

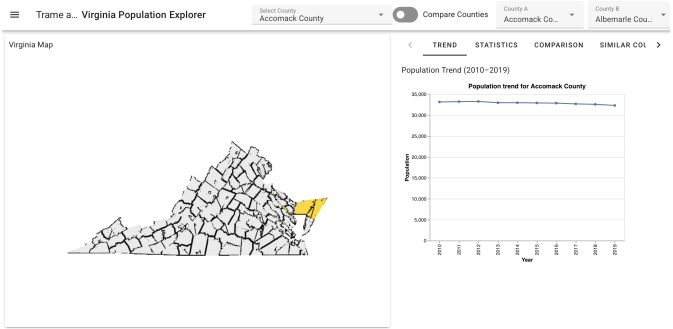


Fig. 3: Interactive system showing synchronized VTK map and Vega-Altair population trend view for the selected county.

5.2 2010 vs. 2019 Comparison

Figure 4 summarizes the population change from 2010 to 2019. Accomack County shows only minor variation, while many other counties experience much larger shifts. This view gives a quick snapshot of long-term demographic change across the state.

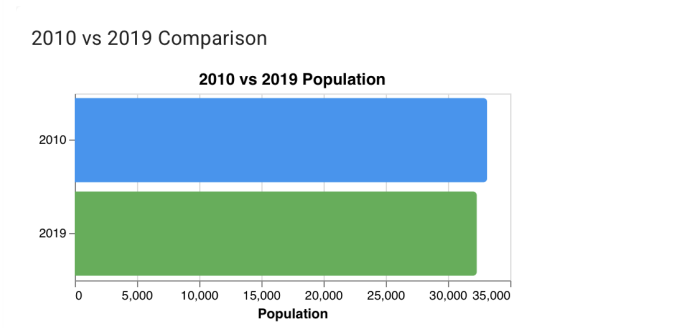


Fig. 4: 2010 vs. 2019 population comparison for the selected county.

5.3 Demographically Similar Counties

Figure 5 displays the counties whose population trends most closely match the selected county. For Accomack, Botetourt and Amherst appear as close matches, revealing shared temporal patterns despite geographic distance.

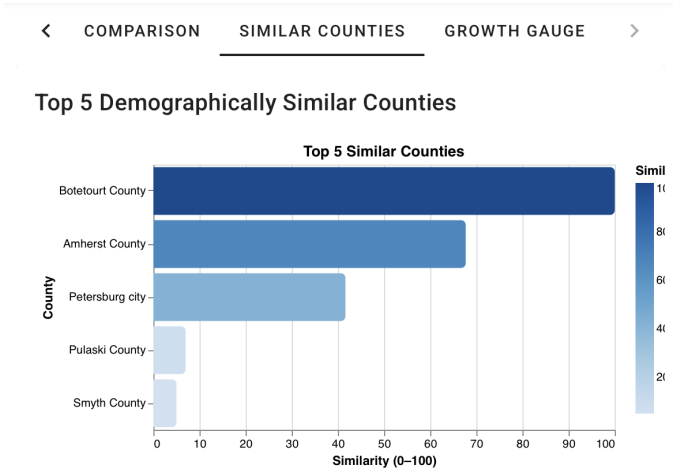


Fig. 5: Top five demographically similar counties based on normalized population trajectories.

5.4 Yearly Population Distribution

Figure 6 shows year-by-year population bars, highlighting small fluctuations or larger shifts. Stable counties have nearly level bars, while growing counties show clear increases across the decade.

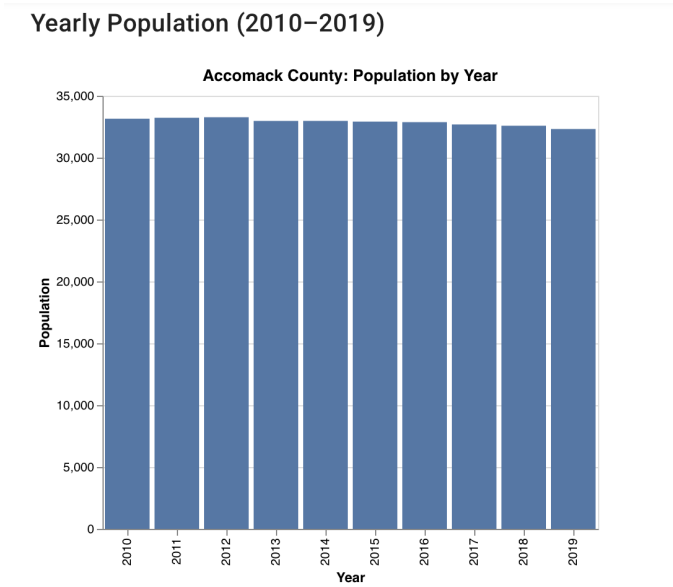


Fig. 6: Yearly population distribution for the selected county (2010–2019).

5.5 Fastest-Growing Counties

Figure 7 shows the top five counties by growth from year 2010 to 2019. Loudoun leads with over 31%, followed by the Fredericksburg and Manassas Park, highlighting Northern Virginia’s strong expansion and statewide growth difference.

Top 5 Fastest Growing Counties (2010–2019)

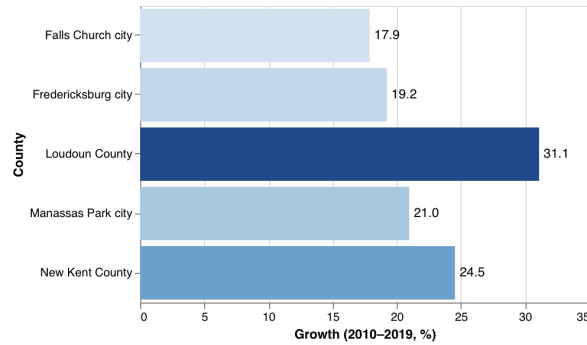


Fig. 7: Top five fastest growing counties in the Virginia from 2010–2019.

5.6 Population Forecasting

Figure 8 projects population to 2030 using simple linear regression. Accomack shows a slight decline while higher growth counties trend upward. The model is simple but provides a helpful first look estimate.

Future Population Forecast

Forecasting population using simple linear regression (2010–2019).

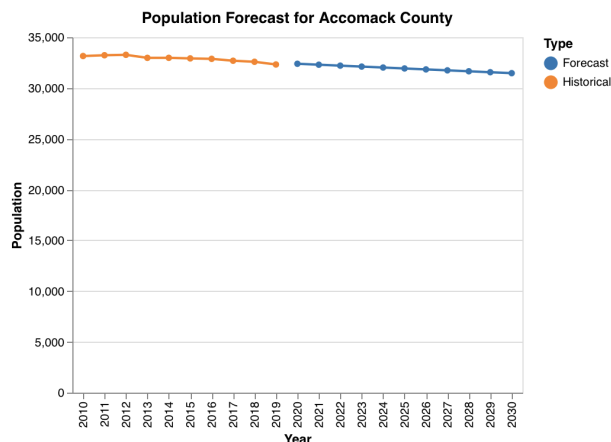


Fig. 8: Population forecast for the selected county from 2020–2030 using the linear regression.

5.7 County Clustering (K-Means)

Figure 9 shows K-Means clusters for all 133 counties. The PCA plot separates fast-growing urban areas from stable or declining rural ones, and the VTK map reveals their spatial distribution.

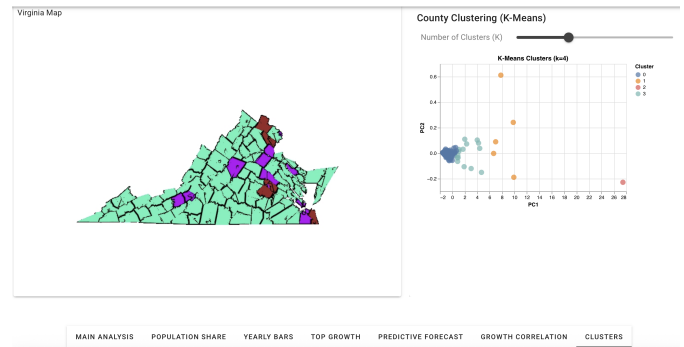


Fig. 9: K-means clustering of Virginia counties using normalized from year 2010 to 2019 population trends.

5.8 Overall Interpretation

The system reveals us clearly the demographic differences across Virginia: The urban areas grows rapidly, while many rural counties stay stable or decline. Combining VTK maps with the Vega Altair charts lets the users move easily from statewide patterns to county level insights.

6 CONCLUSION AND FUTURE WORK

This project presented an interactive system for exploring the Virginia's county level population trends from years between 2010 and 2019. By combining a VTK based map with Vega Altair charts through Trame, we showed how spatial and statistical the views can work together in a single platform. The Users can study temporal changes, compare the counties and find similar regions, while added modules like K-Means clustering and linear regression forecasting reveal deeper patterns and future trends. So overall, this system turns complex demographic data into a clear and accessible exploratory tool.

Future work could include adding more demographic variables like age, migration, education or socioeconomic indicators to support richer analysis and more expressive clustering. The more advanced forecasting methods like ARIMA, Prophet or neural models could improve the long term predictions. The geographic interface could also benefit us from zooming, panning, animations, or timelapse views. Finally, extending the system beyond Virginia to regional or national datasets would broaden its usefulness for comparative demographic studies.

REFERENCES

- [1] J. Han, M. Kamber, and J. Pei. *Data Mining: Concepts and Techniques*. Morgan Kaufmann, 3 ed., 2011. 1, 2
- [2] G. Moreira, M. Lage, N. Ferreira, and F. Miranda. High-level grammars for visualization and visual analytics. In *2023 36th SIBGRAPI Conference on Graphics, Patterns and Images (SIBGRAPI)*, pp. 270–275. IEEE, 2023. 2
- [3] K. Moreland, T. M. Athawale, V. Bolea, M. Bolstad, E. Brugger, H. Childs, et al. Visualization at exascale: Making it all work with vtk-m. *The International Journal of High Performance Computing Applications*, 38(5):508–526, 2024. 1, 2
- [4] A. Satyanarayan, D. Moritz, K. Wongsuphasawat, and J. Heer. Vega-lite: A grammar of interactive graphics. *IEEE Transactions on Visualization and Computer Graphics*, 23(1):341–350, 2017. 2
- [5] W. J. Schroeder, K. M. Martin, and W. E. Lorensen. *The Visualization Toolkit: An Object-Oriented Approach to 3D Graphics*. Prentice-Hall, Upper Saddle River, NJ, USA, 1998. 1, 2
- [6] J. Zong, J. Pollock, D. Wootton, and A. Satyanarayan. Animated vega-lite: Unifying animation with a grammar of interactive graphics. *IEEE Transactions on Visualization and Computer Graphics*, 29(1):149–159, 2023. 2